

Open-source gravity and magnetic forward model of ellipsoids

Santiago Soler (ssoler@eoas.ubc.ca), Kelly Baker, Lindsey Heagy

Summary

We present an **open-source Python** implementation of the **analytic solutions** of the **gravity** and **magnetic** fields of **ellipsoids** with arbitrary orientations. It can compute:

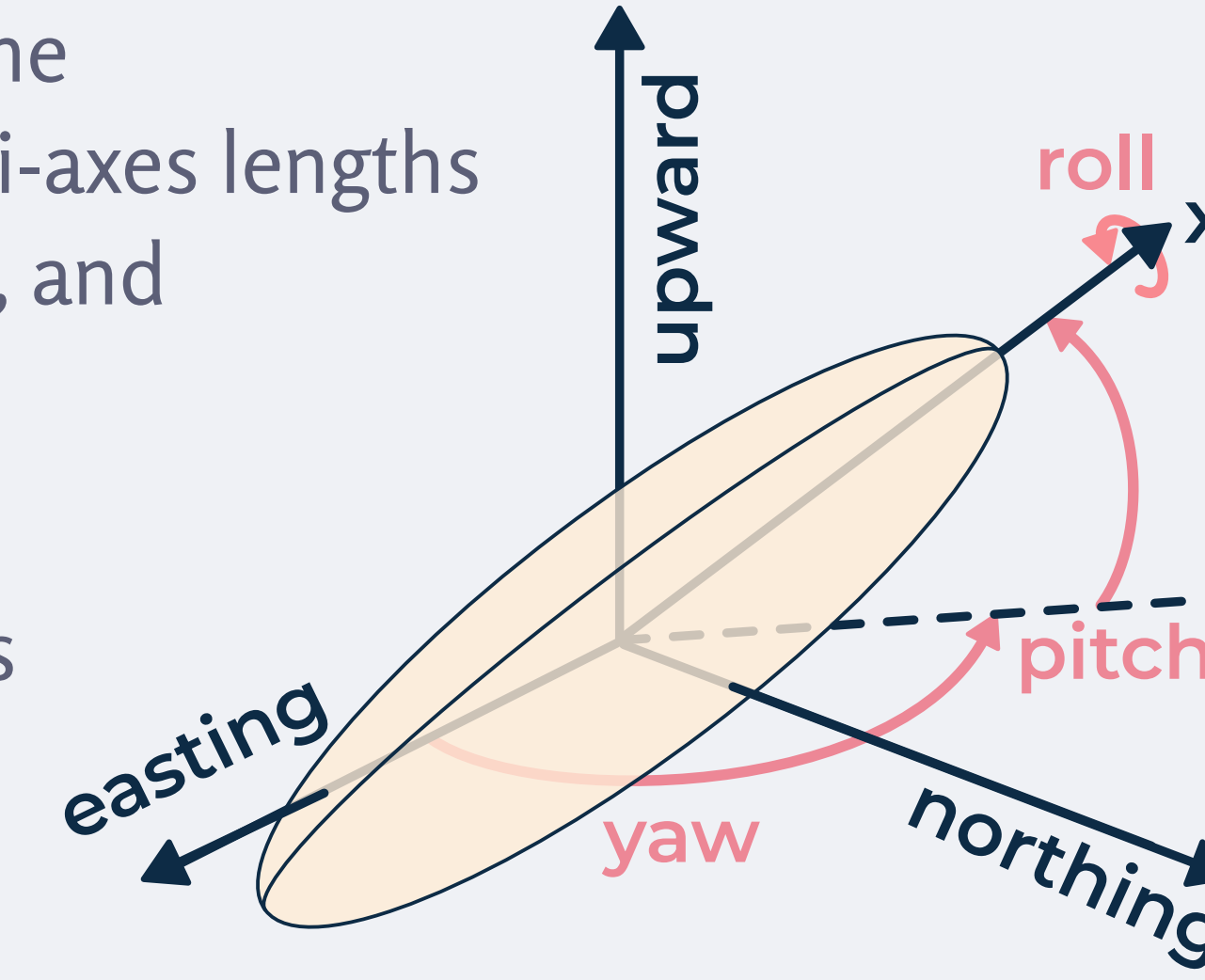
- the **gravity acceleration** due to ellipsoids with homogeneous **density**; and
- the **magnetic field** due to ellipsoids with magnetic **susceptibility** and **remanent magnetization**, accounting for **self-demagnetization** effects.

The code has been included in **Harmonica**, an open-source Python library, part of **Fatiando a Terra**.

Ellipsoid

Ellipsoids can be represented through the **Ellipsoid** class by specifying its semi-axes lengths (**a**, **b**, **c**), the coordinates of its **center**, and 3 rotation angles (**yaw**, **pitch**, **roll**).

We can assign physical properties like its **density**, **susceptibility**, and **remanent_mag** vector.

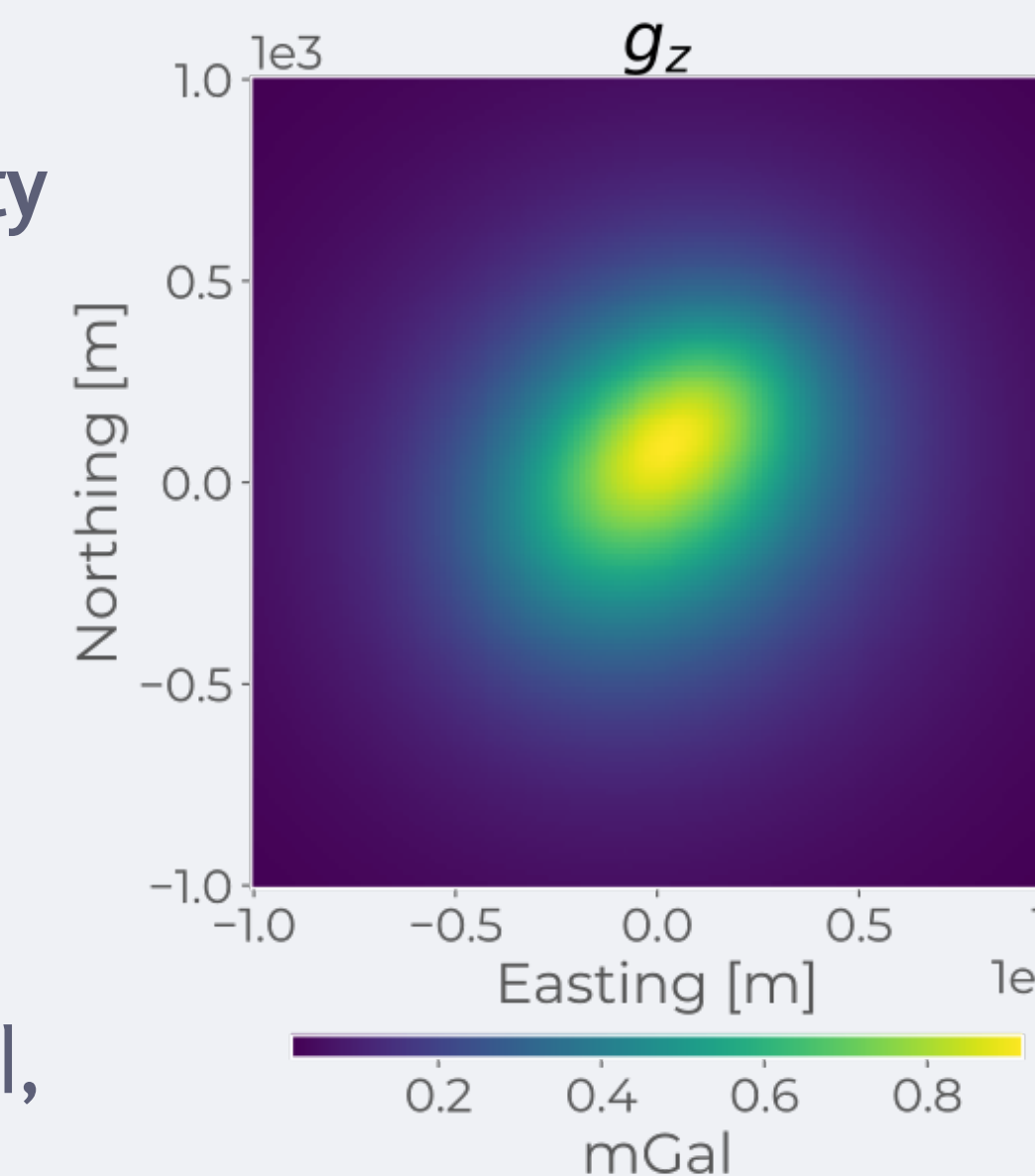


Gravity

The **hm.ellipsoid_gravity** function implements the analytic solution for the **gravity acceleration** vector of an homogeneous ellipsoid:

$$\mathbf{g}(\mathbf{p}) = \nabla U(\mathbf{p}) = G\rho \nabla \int_{\Omega} \frac{d\mathbf{q}}{|\mathbf{p}-\mathbf{q}|} = G\rho \nabla f(\mathbf{p})$$

The **f(p)** can be expressed as an integral in ellipsoidal coordinates. Its gradient components have analytic solutions for triaxial, oblate and prolate ellipsoids.



Magnetics

The **hm.ellipsoid_magnetic** function implements the analytic solution for the **magnetic field** an ellipsoid with magnetic susceptibility and uniform remanent magnetization. Its magnetic field can be expressed as:

$$\mathbf{B}(\mathbf{p}) = \mu_0 \mathbf{H}(\mathbf{p}) \quad (\text{external})$$

$$\mathbf{B}(\mathbf{p}) = \mu_0 [\mathbf{H}(\mathbf{p}) + \mathbf{M}] \quad (\text{internal})$$

magnetization vector

The **H** field due to the ellipsoid can be expressed as:

$$\mathbf{H}(\mathbf{p}) = -\mathbf{N}(\mathbf{p})\mathbf{M}$$

demagnetization tensor

$$\mathbf{N}(\mathbf{p}) = -\frac{1}{4\pi} \nabla \nabla f(\mathbf{p}) = -\frac{1}{4\pi} \begin{bmatrix} \partial_{xx}^2 f & \partial_{xy}^2 f & \partial_{xz}^2 f \\ \partial_{xy}^2 f & \partial_{yy}^2 f & \partial_{yz}^2 f \\ \partial_{xz}^2 f & \partial_{yz}^2 f & \partial_{zz}^2 f \end{bmatrix}$$

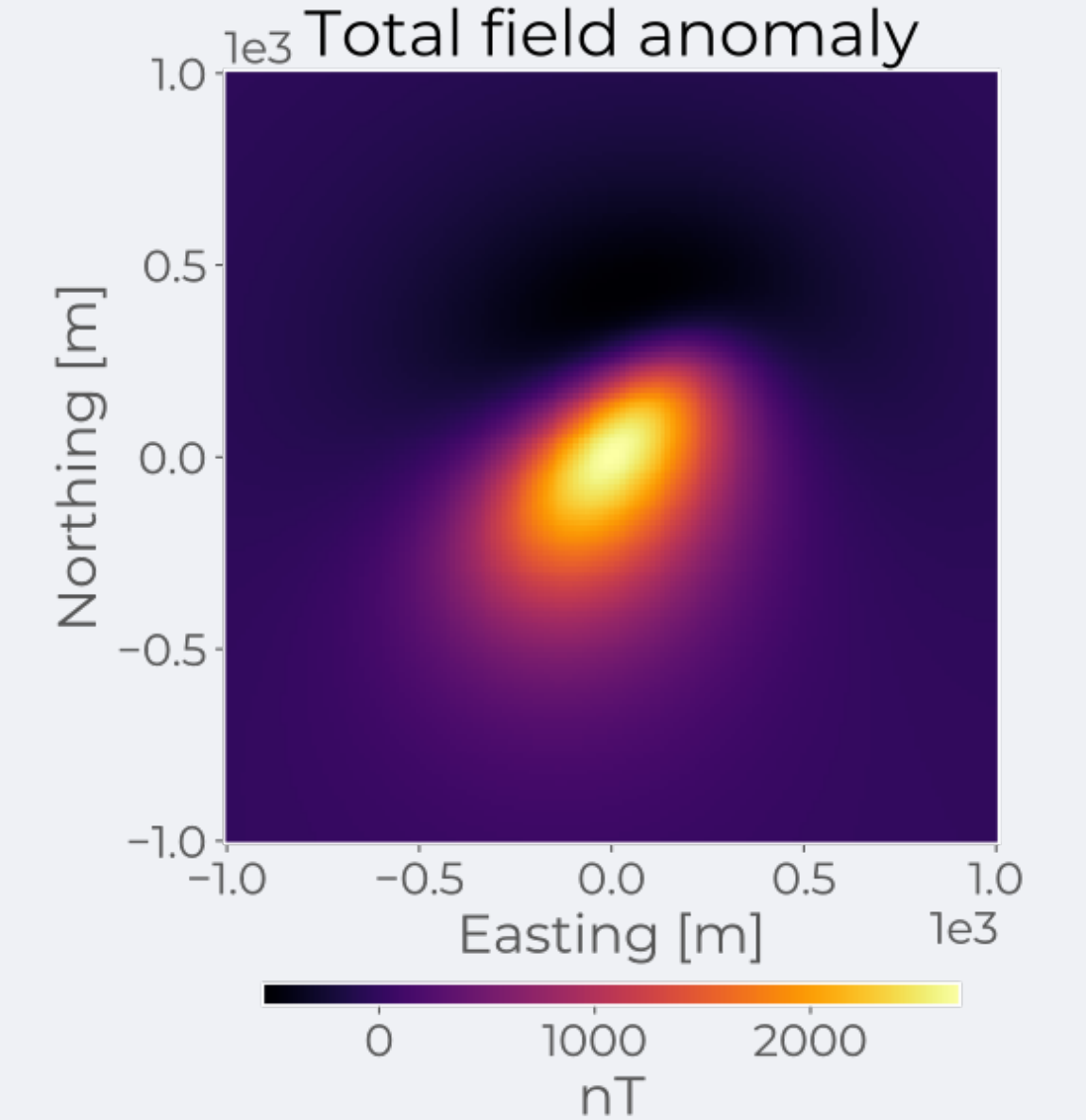
Hessian of f(p)

Analytic solutions for the **second derivatives** of exist for triaxial, oblate, and prolate ellipsoids. Therefore, it's possible to calculate the **self-demagnetization tensor**.

The magnetization vector of the ellipsoid can be obtained by solving a linear system:

$$\mathbf{M} = [\mathbf{I} + \chi \mathbf{N}_{\text{internal}}]^{-1} [\chi \mathbf{H}_0 + \mathbf{M}_{\text{rem}}]$$

susceptibility demagnetization tensor on internal point inducing field remanent magnetization



```
example.py

import harmonica as hm

# Define location of observation points
coordinates = ...

# Generate a triaxial ellipsoid with density and susceptibility
ellipsoid = hm.Ellipsoid(
    a=500, b=300, c=200, # semi-axes lengths
    center=(0.0, 0.0, -300.0), # location of its center
    yaw=45.0, # yaw angle
    pitch=15.0, # pitch angle
    roll=60.0, # roll angle
    density=200.0, # in kg/m3
    susceptibility=0.3, # in SI units
)

# Compute gravity acceleration
gx, gy, gz = hm.ellipsoid_gravity(coordinates, ellipsoid)

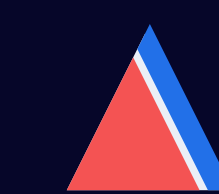
# Define inducing field
intensity = 49_171 # in nT
inclination, declination = 64.8, 5.3
inducing_field = hm.magnetic_angles_to_vec(
    intensity, inclination, declination
)

bx, by, bz = hm.ellipsoid_magnetic(coordinates, ellipsoid, inducing_field)
```

The ellipsoids code has been included in Harmonica: an open-source Python library part of the Fatiando a Terra project.



Fatiando a Terra
Open-source Python tools for Geophysics
www.fatiando.org



Harmonica
fatiando.org/harmonica

Most of the development was carried out by Kelly Baker as part of a Summer Research Project at GIF.



Tip!

We can pass multiple ellipsoids to the forward functions:

```
# Build a collection of ellipsoids
ellipsoids = [hm.Ellipsoid(...), hm.Ellipsoid(...)]

# Compute their gravity acceleration
gx, gy, gz = hm.ellipsoid_gravity(coordinates, ellipsoids)
```

